

25.1 Question 1 (5 marks)

**LAB
ACTIVITY**

25.1.1: Question 1 (5 marks)

 **Full screen**

0 / 5



In this exercise you are required to implement the C++ function `getPrimesInRange` that returns a vector of all prime numbers within a specified range. The details for the function are provided below

You are provided with the following files:

- `main.cpp` - provided for you to use as you develop your code
 - `Task.h` - header file (read only)
 - `Task.cpp` - you will write your assessed C++ code in this file
 - `utilities.h` - (hidden) contains `printVector` for use as you develop your code
-

Function Details

Implement the following function:

```
std::vector<int> getPrimesInRange(int lower_bound, int  
upper_bound);
```

The function has the following input parameters are

- `lower_bound`: The starting point of the range (inclusive).
- `upper_bound`: The end point of the range (inclusive).

The function should return:

- A vector of integers containing all the primes that are within the range [`lower_bound`, `upper_bound`].
 - **NOTE** the primes should be in ascending order (as shown in the example below)
- If no multiples are found or the inputs are invalid (e.g., `lowerbound > upperbound`), return an empty vector.

HINT

- you are recommended to implement a **helper** function `bool isPrime(int num)` in `Task.cpp` (see notes in the file) to determine if a number is a prime
 - use the following steps:
 - Handle base case: any number less than or equal to 1 is **not prime**
 - Loop through possible divisors: from `i=2` to square root of `num` i.e. to `i*i <= num`
 - inside the loop check for divisibility i.e. if `num % i == 0`
-

Example

For the following function call:

```
getPrimesInRange(10, 50);
```

The result vector should contain the following values:

```
{11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47}
```

[Open new tab](#)[Dock](#)

▶ Run

History

Tutorial

```
1  #include <iostream>
2  #include "Task.h"
3  #include "utilities.h" // (hidden) contains printVector for testing
4
5
6  int main() {
7      std::vector<int> primes = getPrimesInRange(10, 50);
8      printVector(primes);
9
10     return EXIT_SUCCESS;
11 }
```

DESKTOP CONSOLE



Model Solution ^

25.2 Question 2 (5 marks)

**LAB
ACTIVITY**

25.2.1: Question 2 (5 marks)

 **Full screen**

0 / 5



In this exercise you are required to implement the C++ function `extractUniqueWords` that iterates over a 2D vector of strings and builds a 1D vector containing all unique words from the rows and columns.. The details for the function are provided below

You are provided with the following files:

- `main.cpp` - provided for you to use as you develop your code
- `Task.h` - header file (read only)
- `Task.cpp` - you will write your assessed C++ code in this file

- `utilities.h` - (hidden) contains `printVector` for use as you develop your code
-

Function Details

Implement the following function:

```
std::vector<std::string> extractUniqueWords(  
    const std::vector<std::vector<std::string>>&  
    wordMatrix);
```

©zyBooks 04/15/26 10:19 3016288

The function has the following input parameter

- `wordMatrix`: A 2D vector of strings, where each inner vector represents a row of words.

The function should return:

- A vector of strings containing all unique words from the 2D vector.
- **NOTE**
 - Words should appear only once in the result, even if they are present in multiple rows or columns.
 - The order of words in the resulting vector **does not matter**

HINT: you are recommended to implement a **helper** function (in `Task.cpp`) that:

- checks if the return vector contains a given word
 - if it **doesn't contain the word** then that word should be pushed back onto the return vector
 - if it **does contain the word** then that word should **not** be pushed back onto the return vector
-

Example

For the following:

```
std::vector<std::vector<std::string>> wordMatrix = {  
    {"apple", "banana", "apple"},  
    {"orange", "banana", "grape"},  
    {"grape", "apple", "melon"}  
};  
extractUniqueWords(wordMatrix);
```

©zyBooks 04/15/26 10:19 3016288
Dutt Patel
EDUCOSC2802Semester12026

The result vector should contain the following (order does not matter):

```
{"apple", "banana", "orange", "grape", "melon"}
```

[Open new tab](#)[Dock](#)

 ▶ Run

History Tutorial

1

DESKTOP CONSOLE 







Model Solution

25.3 Question 3 (5 marks)

**LAB
ACTIVITY**

25.3.1: Question 3 (5 marks)

 **Full screen**

0 / 5



In this exercise you are required to implement the C++ code for an `AFLTeam` class that includes attributes for the team's name, points, games won, and games lost. You will also be asked to overload the `<<` operator as a member function to format and display this information. The details for the function are provided below

You are provided with the following files:

- `main.cpp` - provided for you to use as you develop your code
- `Task.h` - header file (read only)
- `Task.cpp` - you will write your assessed C++ code in this file
- `utilities.h` - (hidden) contains `printVector` for use as you develop your code

Class Details

You must implement the `AFLTeam` class. Define the `AFLTeam` Class, with the following:

- Private member variables
 - `name` (`std::string`): The name of the team.
 - `points` (`int`): The team's total points in the league.
 - `gamesWon` (`int`): The number of games the team has won.
 - `gamesLost` (`int`): The number of games the team has lost
- Implement the constructor to correctly initialise all four private data members (in file `Task.cpp`)

```
AFLTeam(const std::string& name, int points, int gamesWon, int gamesLost);
```

Overload operator<<

You must implement (in file `Task.cpp`) a non-member `operator<<` overload to allow objects of class `AFLTeam` to be printed using `std::cout`

```
std::ostream& operator<<(std::ostream& os, const AFLTeam& team);
```

Example

For the following:

```
AFLTeam team("West Coast Eagles", 90, 0, 0);  
std::cout << team << std::endl;
```

should output:

```
Team: West Coast Eagles  
Points: 90  
Games Won: 0  
Games Lost: 0
```

a representation of the string, showing escape characters (e.g. \n) is:

```
"Team: West Coast Eagles\nPoints: 90\nGames Won: 0\nGames Lost:  
0\n"
```

NOTE all separating spaces are **SINGLE** whitespace characters ie ' '

[Open new tab](#)[Dock](#)



▶ Run

History

Tutorial

```
1  #include "Task.h"
2  #include <iostream>
3
4  int main() {
5      AFLTeam team("West Coast Eagles", 90, 0, 0);
6      std::cout << team << std::endl;
7
8      return EXIT_SUCCESS;
9  }
```

DESKTOP

CONSOLE**Model Solution** ^

25.4 Question 4 (5 marks)

**LAB
ACTIVITY**

25.4.1: Question 4 (5 marks)

 **Full screen**

0 / 5



Write the C++ function as detailed below. You are provided with the following files:

- `main.cpp` - sample code to help you develop your program. This is not used to test your code.
 - `task.h` (read only header file).
 - `task.cpp` - you will write your assessed C++ code in this file
-

Task

Implement the following function:

```
std::vector<std::string> collectItemsByLabel(  
    std::stringstream& sStream,  
    const std::string& targetLabel);
```

The function has the following parameters:

- `sStream`: A `std::stringstream` containing labeled items. Each label and item are separated by a single space character ' ', and each line corresponds to one labeled item. For example, shown below, each line (ie separated by `\n`) contains:
 - A label (e.g., "electronics", "furniture"); and
 - An item (e.g., "Laptop", "Smartphone", "Table" and "Chair")
 - NOTE: you can assume all input will be well formed (ie contain label, followed by item followed by `\n` as shown below)

```
std::stringstream inputStream(  
    "electronics Laptop\nelectronics Smartphone\nfurniture  
Table\nfurniture Chair");
```

- `targetLabel`: a `std::string` specifying the label of interest (e.g., "electronics").

The Function Should:

- process each line in the `sStream` to extract the `label` and `item`.
 - Construct a `std::vector<std::string>` containing only the items associated with the specified label (ie the `targetLabel`).
 - Return the vector of items that match the `targetLabel`
-

Example

for the following:

```
std::stringstream inputStream(  
    "electronics Laptop\nelectronics Smartphone\nfurniture  
Table\nfurniture Chair");  
collectItemsByLabel(inputStream, "electronics");
```

should return a vector containing the following elements (order doesn't matter):

```
{"Laptop", "Smartphone"}
```

[Open new tab](#)[Dock](#)

Run

1

History

Tutorial

DESKTOP

CONSOLE



Model Solution ^

25.5 Question 5 (10 marks)

**LAB
ACTIVITY**

25.5.1: Question 5 (10 marks)

Full screen

0 / 10



Complete the provided skeleton C++ code as detailed in the tasks below which relate to a Sports Team information system. You are provided with the following:

- `main.cpp`: this contains code which you can use to develop your code. It will not be used in the unit tests.
- Base class:
 - `Team.h`: (header file) you will **write assessed C++ code in this file**

- `Team.cpp`: provided for you and contains the member function definitions and implementation details. You can use this code (the file/implementation is hidden)
 - Derived class:
 - `SoccerTeam.h`: (header file) you will **write assessed C++ code in this file**
 - `SoccerTeam.cpp`: you will **write assessed C++ code in this file**
 - `TeamList.h`: header file (read only)
 - `TeamList.cpp`: **you will write assessed C++ code in this file**
 - `utilities.h` - includes `escapeString` which you can use to test as you develop your code (the file/implementation is hidden)
-

Task 1

Given the base class `Team` (see file `Team.h`), complete the derived class `SoccerTeam` with the constructor below. Note: Your `SoccerTeam` constructor must initialise the base-class part of the object by explicitly calling the `Team` constructor in the member-initialiser list.

```
SoccerTeam(std::string name = "No name", std::string coach = "No  
coach",  
           int played = 0, int won = 0, int lost = 0, int drawn =  
0);
```

- This constructor has 6 parameters: `name`, `coach`, `played`, `won`, `lost` and `drawn`.
 - Use the parameters `name`, `coach`, and `played` when calling the `Team` constructor
 - The other parameter values, `won`, `lost` and `drawn`, should be assigned to the respective data members.
-

Task 2

Complete the `SoccerTeam` member function

```
std::string formatInfo();
```

which creates and returns a formatted `std::string` as follows (and shown in the example below):

- all member fields in the class are separated by ' | ' followed by a single space character ' '
- `team` and `coach` are left justified in a field width of 19
 - This means that there are exactly 20 spaces between each ' | ' character)
- all other fields are left justified in a field width of 9 (ie exactly 10 spaces between each ' | ' character).
- The overall length of the string is 87:
 - the sum of each field + 7 separator characters ' | ' followed by single space ' '

Example

For the following `SoccerTeam` object and function call:

```
SoccerTeam nufc("Newcastle United", "Eddie Howe", 32, 15, 12, 5);
nufc.formatInfo();
```

the method should return the following string:

```
"| Newcastle United   | Eddie Howe           | 32       | 15       |
12         | 5         |"
```

Task 3

Given `TeamList.h` complete the member function

```
std::string formatTeamList();
```

Details are:

- it has no input parameters
- it returns a formatted string of the objects in `TeamList` (class data member) which can be either:
 - `Team` base class objects
 - `SoccerTeam` derived class objects
- the `Team` objects should be formatted using the base class member function `formatInfo()`
- the `SoccerTeam` objects should be formatted using its member function `formatInfo()`
 - i.e. override the base class function). The first 2 lines of the string should print as:

Also note the following additional parts of the formatted string (as shown in the example below):

- **Header lines:** the first 2 lines of the formatted team list string when printed out should be

```
| Name           | Coach           | Played | Won |
Lost      | Drawn      |
=====
=====
```

- Line Separator: after each row, either `Team` (base objects) or `SoccerTeam` (derived objects) the following line:

```
-----
-----
```

NOTE

- You will need to use polymorphism and virtual functions to allow for function overriding
- You will need to make changes in the following files: `Team.h` and `SoccerTeam.h`

Example

for the following:

```
TeamList teamList;
teamList.addTeam(&nufc);
```

```
teamList.addTeam(&hawks);
teamList.addTeam(&spurs);
teamList.addTeam(&demons);
```

the following:

```
teamList.formatTeamList()
```

©zyBooks 04/15/26 10:19 3016288

should return a formatted table, which when output is as follows:

```
| Name                | Coach                | Played | Won          |
Lost      | Drawn      |
=====
=====
| Newcastle United   | Eddie Howe          | 32      | 15          |
12          | 5          |
-----
-----
| Hawthorn           | Sam Mitchell        | 15
|
-----
-----
| Tottenham Hotspurs | Ange Postecoglou    | 32      | 18          | 8
| 6          |
-----
-----
| Melbourne          | Simon Goodwin       | 5
|
-----
-----
```

NOTE the actual string that is returned by the Base Class Team function `formatInfo` will need padding at the end to match the required format of the table. For example, for the following:

```
Team hawks("Hawthorn", "Sam Mitchell", 15);
std::string test = hawks.formatInfo();
std::cout << escapeString(test) << "\n";
```

©zyBooks 04/15/26 10:19 3016288

the string returned (printed using `escapeString`):

```
"| Hawthorn          | Sam Mitchell        | 15          "
```

But to fit in the formatted table, this needs padding at the right hand end and the addition of ' | ' (so that the total length of the string is 87 characters), so that it outputs as follows:

Hawthorn	Sam Mitchell	15

[Open new tab](#)[Dock](#)

Run

[History](#)

Tutorial

```
1  #include <iostream>
2  #include "Team.h"
3  #include "SoccerTeam.h" // the implementation of this class is provided for
4  #include "TeamList.h"
5  #include "utilities.h" // Includes escapeString(std::string) for use to dev
6
7  int main()
8  {
9      SoccerTeam nufc("Newcastle United", "Eddie Howe", 32, 15, 12, 5);
10     SoccerTeam spurs("Tottenham Hotspurs", "Ange Postecoglou", 32, 18, 8, 6
11     Team hawks("Hawthorn", "Sam Mitchell", 15);
12     Team demons("Melbourne", "Simon Goodwin", 5);
13
14     std::string test_str = nufc.formatInfo();
15     std::cout << test_str << " " << test_str.length() << "\n\n";
16
17     TeamList teamList;
18     teamList.addTeam(&nufc);
19     teamList.addTeam(&hawks);
20     teamList.addTeam(&spurs);
21     teamList.addTeam(&demons);
22 }
```

DESKTOP

CONSOLE**Model Solution** ^

25.6 Question 6 (10 marks)

**LAB
ACTIVITY**

25.6.1: Question 6 (10 marks)

 **Full screen**

0 / 10



Complete the provided skeleton C++ code as detailed in the tasks below which relate to an online shopping application. The following files are supplied:

- `main.cpp` - this contains code to use while you develop your code. This will **not** be used for testing your code.
- `Basket.h` (read only)
- `Basket.cpp` - you will **write your assessed C++ code for Tasks 1-4 in this file**
- `Item.h` and `Item.cpp` (read only) - you are **required to use this** to complete the `Basket` class

Your completed code will be tested with unit tests that dynamically create objects of the `Basket` class, that contain dynamically allocated objects of the `Item` class. Examples are provided in `main.cpp` (see lines 19-23) as follows:

```
Basket *basket = new Basket();  
basket->items.push_back(new Item("milk", 3.99, 1));  
basket->items.push_back(new Item("eggs", 7.00, 1));  
basket->items.push_back(new Item("cheese", 5.50, 2));
```



```
std::cout << "Count of dynamically allocated Item objects: ";  
std::cout << Item::objectCount << "\n";
```

This will dynamically allocate a `Basket` object on the heap, dynamically allocate 3 `Item` objects, and add pointers to them in the `items` vector inside the `Basket` object. Then the number of `Item` objects is output.

Task 1

Complete the `Basket` class member function `float Basket::get_basket_total()` as follows:

- the function has no parameters
- the function returns a `float`, the total cost for all the items in the basket
 - the cost for each item in the basket is `(price * quantity)`

Example 1:

For the following basket:

```
Basket *basket = new Basket();  
basket->items.push_back(new Item("milk", 3.99, 1));  
basket->items.push_back(new Item("eggs", 7.00, 1));  
basket->items.push_back(new Item("cheese", 5.50, 2));  
basket->get_basket_total();
```

the function call should return

```
21.99
```

Example 2:

An empty basket should return: 0

Task 2

Complete the `Basket` class member function as described below

```
void Basket::add_item(std::string name, float price, int quantity)
```

The function has 3 parameters:

- a `std::string`, the name of the item to be added
- a `float`, the price of a single quantity of the item
- an `int`, the quantity to be added

There are 2 cases to handle in your code:

- no `Item` object with the same name exists in the `items` vector in this basket. In this case:
 - a new `Item` object is dynamically created, with corresponding values of `name`, `price` and `quantity`
 - **NOTE:** this will increment the static data member `Item::objectCount`
 - a pointer to this `Item` object is added to the data member, the vector of `items`
- an `Item` object with the same name exists in the `items` in this basket. In this case:
 - the `quantity` of this item is increased by the input `quantity`

Example 1

For the following:

```
Basket *basket = new Basket();  
basket->items.push_back(new Item("milk", 3.99, 4));
```

will create an object instance of `Basket` with a single item. Then the following function call to add an item will

```
basket->add_item("milk", 3.99, 1);
```

- increase the value of `quantity` to 5
- leave `Item::objectCount` unchanged (i.e. no new `Item` object created)

Example 2

For the following:

```
Basket *basket = new Basket();  
basket->items.push_back(new Item("milk", 3.99, 4));  
basket->add_item("cheese", 6.5, 1);
```

the function call will:

- create a new `Item` object with data members:
 - `name` set to "cheese"
 - `price` set to 6.5
 - `quantity` set to 1
 - increment `Item::objectCount` by 1 (this is done automatically by the provided code (`Item.h` and `Item.cpp`), when you create a single new `Item` object)
-

Task 3

Complete the `Basket` class member function as follows:

```
void Basket::remove_item(std::string name, int quantity)
```

- the function has 2 parameters:
 - a `std::string`, the name of the `Item` object to remove from the basket
 - an `int`, the quantity of the `Item` to remove
- the function operates as follows:
 - it finds the `Item` object with the same name in `items` (`Basket` data member).
NOTE your code will only be tested with names that are found in `items`.
 - Then the function
 - reduces the `quantity` data member by the input `quantity`
 - if this reduces the `quantity` to **zero or less** then:
 - that `Item` object **must be deallocated** (ie using `delete`)
 - remove the pointer to that `Item` object from `items` (ie the `Basket` vector data member)

Example 1

for the following:

```
Basket *basket = new Basket();  
basket->items.push_back(new Item("cheese", 6.5, 2));
```

the function call `basket->remove_item("cheese", 1)` will:

- reduce the `quantity` of the `Item` object with name "cheese" from 2 to 1

Example 2

for the following

```
Basket *basket = new Basket();  
basket->items.push_back(new Item("milk", 3.99, 4));
```

the function call `basket->remove_item("milk", 4)` will:

- deallocate the `Item` object with name "milk"
 - this will decrement by 1 the value of `Item::objectCount`
- remove the pointer to the `Item` with name "milk" from the `items` in the `Basket`
 - HINT:** to do this you can use `erase(iteratorPosition)` which removes the vector element from `iteratorPosition`. Elements from higher positions in the vector are shifted back to fill the gap. Vector size is decremented. Review zybooks section 14.13 for further details.

Task 4

You are asked to complete the destructor for the `Basket` class:

```
Basket::~~Basket()
```

Your function should:

- deallocate all `Item` objects that are pointed to from the `items` data member of the `Basket` object
- remove the `items` data member of the `Basket` object

Example

For the following:

```
Basket *basket = new Basket();  
basket->items.push_back(new Item("milk", 3.99, 4));  
basket->items.push_back(new Item("eggs", 7.00, 1));  
delete basket;  
std::cout << "Count of dynamically allocated Item objects: ";  
std::cout << Item::objectCount << "\n";
```

should output

```
Count of dynamically allocated Item objects: 0
```

[Open new tab](#)[Dock](#)

5

 Run

 History

Tutorial

1

DESKTOP

CONSOLE







Model Solution ^

